

Travail collaboratif & documentation technique

SUPPORT DE COURS

Séance 2 · Le backlog, puis relire une pull request

Matière	Travail collaboratif & documentation technique · Ycode A2627_4727
Année scolaire	2026-2027
Formateur	Luc Charloiseau
Groupe cible	Bachelor 2 Informatique, IA & Data et Cybersécurité
Dernière mise à jour	30/09/2026

Prérequis : séance 1

Deux temps aujourd'hui. Avant la pause, vous finissez de reprendre le projet : le backlog laissé par l'équipe précédente devient des issues propres, et vous apprenez à signaler un blocage.

Après la pause, vous relisez le code des autres. Cinq pull requests attendent sur le dépôt modèle. Certaines sont bonnes. Une autre permet de lire les mots de passe de tous les utilisateurs.

Ce que vous saurez faire en fin de séance

Rédiger une issue d'évolution avec des critères d'acceptation vérifiables.

Trier un backlog : découper, prioriser, étiqueter, assigner.

Signaler un blocage de façon concise et actionnable.

Relire une PR avec méthode : correction, sécurité, performance, lisibilité.

Écrire des commentaires constructifs, étiquetés, bloquants ou non.

Choisir entre Comment, Approve et Request changes.

Déroulé

Quand	Contenu	Durée
Vendredi 2 octobre (3 h)	Intro : le backlog, l'évolution et ses critères, trier (parties 1 et 2)	10 min
	Exercice 1 : le backlog en issues (<i>noté</i>)	45 min
	Point de cours : signaler un blocage (partie 3)	5 min
	Exercice 2 : un message de blocage	10 min
	Pause	30 min
	Intro : l'écran bleu de CrowdStrike, la démo d'injection, relire une PR (parties 4 à 6)	15 min
	Exercice 3 : le défi, trouver la PR qui contient une injection SQL	35 min
	On présente : la révélation du défi	10 min
	Marge : finir l'exercice en cours, questions ; s'il reste du temps, le quiz des issues, pour s'entraîner au quiz final	20 min

Comment lire ce support. Vous n'avez pas à tout lire d'avance. Juste avant chaque exercice, l'enseignant présente la partie qui sert ; pendant l'exercice, gardez cette partie sous les yeux. Le reste sert de référence. L'exercice 1 se fait sur votre dépôt, et il est noté. L'exercice 3 entraîne votre œil avant de relire le code de vos coéquipiers, en séance 3. Un exercice qui déborde prend sur la marge de fin.

Ouverture : le backlog de l'équipe précédente

Hier, vous avez réécrit un signalement flou, et un autre membre a reproduit le bug sans vous parler.

L'équipe précédente a laissé d'autres demandes : cinq dans le backlog, et deux anomalies signalées à l'oral que personne n'a écrites. Certaines sont des **bugs** : on les reproduit, puis on les décrit au format de la séance 1. Les autres sont des **évolutions** : on n'y décrit pas un symptôme, mais un résultat attendu, qu'il faut pouvoir vérifier.

Et pendant que vous travaillez, il arrivera que vous bloquiez. La partie 3, présentée après l'exercice 1, dit comment le signaler, pour que quelqu'un puisse vous aider en cinq minutes.

Partie 1 : une issue d'évolution

Le format

Rubrique	Contenu
Titre	le résultat attendu
Contexte	le problème de l'utilisateur
Besoin	ce qu'il faut rendre possible
Critères d'acceptation	une liste vérifiable
Hors périmètre	ce qui n'est pas demandé

Les critères d'acceptation sont la rubrique décisive :

- [] `'GET /api/v1/jeux/genres'` renvoie les genres triés, sans doublon
- [] un catalogue vide renvoie `'[]'` avec un statut 200
- [] la route est couverte par des tests

« Améliorer la recherche » ne se termine jamais. Un critère se vérifie par oui ou par non.

L'issue #1 du dépôt modèle est l'exemple complet. Lisez-la.

Pour un bug, le format ne change pas : c'est celui de la séance 1, partie 3. Titre qui décrit le symptôme, étapes depuis un état connu, attendu, observé **copié**, environnement.

Partie 2 : trier le backlog

Une fois écrite, une issue se trie.

Étiqueter. Une étiquette sert à filtrer. Celles que GitHub crée suffisent : `bug`, `enhancement` pour une évolution, `documentation`, `question`. N'en créez pas d'autres.

S'assigner, au moment où l'on commence. Assignée veut dire « quelqu'un y travaille en ce moment ». C'est ce qui évite que deux personnes corrigent le même bug.

Relier à la PR qui la fermera. `Closes #12` dans la description de la PR ferme l'issue à la fusion. GitHub reconnaît `Closes`, `Fixes`, `Resolves`, **uniquement en anglais** : `Corrige #12` ne ferme rien.

Découper ce qui est trop gros. Une issue doit tenir dans une PR relisible en moins de trente minutes. Au-delà, on découpe **par résultat**, pas par fichier :

- [] #21 Enregistrer l'avis d'un utilisateur
- [] #22 Afficher la moyenne des avis sur la fiche
- [] #23 Masquer un avis inapproprié

Une issue « créer la table des avis » n'apporte rien de vérifiable à l'utilisateur. La priorité, enfin, va à ce qui gêne le plus de monde, le plus souvent. Une faille de sécurité passe avant tout.

✓ Exercice 1 : le backlog en issues (45 min)

À relire avant : les parties 1 et 2.

Le but : cinq issues, une par demande, qu'un autre pourrait traiter sans vous parler. Vous ne corrigez rien. Tout se passe sur le dépôt de votre groupe.

Le backlog

Les cinq demandes laissées par l'équipe précédente, telles quelles :

1. **Les recommandés ne marchent plus** : La page des jeux recommandés ne s'affiche plus du tout. Erreur.
2. **problème modif note** : quand je modifie la note d'un jeu ça me dit que le titre existe déjà ??? alors que je touche pas au titre
3. **Éditeurs par pays** : Le client voudrait pouvoir voir les éditeurs d'un pays.
4. **filtre années** : Pouvoir filtrer les jeux entre deux années ce serait top (genre les jeux des années 2010).
5. **Plateformes** : Il faudrait gérer les plateformes (PC, Switch, PS5...) sur les jeux. À voir comment on fait, il y a plusieurs façons.

Le TP

1.1 Répartissez les demandes : une ou deux par membre. Chacun s'assigne les issues qu'il écrit.

1.2 Les demandes 1 et 2 sont des **bugs**. Reproduisez-les avant d'écrire, et **copiez** la réponse de l'API. Si vous avez vidé votre base jeudi, relancez d'abord `python scripts/peupler.py` : il recrée le catalogue et le compte administrateur. Pour reproduire :

- demande 1 : appelez `GET /api/v1/jeux/recommandes` ;
- demande 2 : dans `/docs`, connectez-vous avec le bouton **Authorize** (`admin@example.com` / `motdepasse123`), puis modifiez un jeu avec `PATCH /api/v1/jeux/{jeu_id}`, en n'envoyant que la note. Essayez une note normale, puis une note hors de l'échelle de 0 à 10 : laquelle déclenche le message ?

1.3 Les demandes 3 et 4 sont des **évolutions** : le besoin, et trois critères d'acceptation vérifiables par oui ou par non suffisent.

1.4 La demande 5, « Plateformes », a plusieurs solutions possibles : trois lignes suffisent. Dites le besoin, et qu'il faudra d'abord une décision écrite, un ADR (séance 4). Ne tranchez pas.

1.5 Étiquetez chaque issue : `bug` ou `enhancement`.

Partez de ces squelettes : copiez-les dans l'issue, puis remplissez. Le titre va dans le champ *Title* de GitHub.

```
### Contexte
Où, depuis quand, qui est gêné.

### Étapes pour reproduire
1.
2.

### Comportement attendu

### Comportement observé
(la réponse de l'API, copiée telle quelle)

### Environnement
Système, version de Python, SQLite, branche.
```

```
### Contexte
Le problème de l'utilisateur.

### Besoin

### Critères d'acceptation
- [ ]
- [ ]
- [ ]

### Hors périmètre
```

Ce qui n'est pas terminé aujourd'hui se termine avant la date limite : le backlog fait partie du rendu.

Vérification : cinq issues, une par demande, étiquetées et assignées ; les deux bugs ont des étapes et la réponse copiée ; les évolutions ont des critères vérifiables par oui ou par non.

Si vous avez fini : pour aller plus loin

Les signalements oraux. Deux anomalies signalées de vive voix, que personne n'a écrites. Leurs auteurs parlent comme des utilisateurs : retrouvez dans l'API ce que Tom appelle « le bouton jeu précédent » et ce que Léa appelle « le compteur », puis écrivez les deux issues.

Tom, développeur du front : « Sur la fiche de Hollow Knight, le tout premier jeu ajouté au catalogue, le bouton "jeu précédent" m'emmène sur un autre jeu. Normalement, il ne devrait même pas s'afficher. »

Léa, testeuse : « Dans "Mes jeux", je n'ai créé qu'un seul jeu, et la liste en affiche bien un seul. Mais le compteur annonce 9 résultats. »

Le test de l'inconnu. Un autre membre lit une de vos issues et tente de la traiter en la lisant seulement, comme hier sur l'issue des statistiques. S'il doit vous poser une question, complétez l'issue.

Partie 3 : signaler un blocage

Le problème

Deux réflexes, tous deux mauvais.

Attendre en silence, par peur de déranger. Trois jours plus tard, rien n'a avancé et personne ne le savait.

Écrire « help ça marche pas ». L'autre doit mener l'enquête à votre place.

Quand

Au-delà de 30 à 60 minutes sans progrès, on signale. Une heure de votre temps coûte plus cher à l'équipe que cinq minutes de celui d'un collègue.

Comment

1. **Ce que j'essaie de faire**, avec le lien de l'issue
2. **Ce qui bloque** : l'erreur, copiée
3. **Ce que j'ai déjà essayé**
4. **Mon hypothèse**, même incertaine
5. **Ce dont j'ai besoin** : de qui, et pour quand

Avant :

```
help les tests marchent plus
```

Après :

Bloqué sur #4 (filtre par années).

Mon nouveau test échoue avant d'atteindre la route :

```
sqlite3.OperationalError: no such table: jeux
```

Déjà essayé : relancer `peupler.py`, supprimer `jeux.db`. Même erreur.

Hypothèse : mon test n'utilise pas la fixture `client` de `conftest.py`.

@samir tu as écrit les fixtures, tu peux confirmer ? Pas urgent avant demain midi, je passe sur #3 en attendant.

La dernière phrase dit à Samir quand répondre, et à l'équipe que le temps n'est pas perdu.

Où

Dans l'issue concernée, en commentaire qui commence par « Bloqué ». Si c'est pressé, un message court dans le chat, avec le lien.

Et quand vous débloquez quelqu'un, **répondez dans l'issue**. Le prochain qui rencontrera le problème y trouvera la réponse.

Donner le but

« Comment je convertis une chaîne en entier dans un validateur ? »

C'est une question sur **votre solution**. Le vrai besoin était peut-être « accepter `2018` et `"2018"` », ce que Pydantic fait déjà. Donnez l'objectif : on vous indiquera peut-être un chemin plus court.

✓ Exercice 2 : un message de blocage (10 min)

À relire avant : la partie 3.

La situation. Vous venez de cloner le dépôt, et l'API refuse de démarrer. Dans le terminal :

```
pydantic_core._pydantic_core.ValidationError: 1 validation error for Configuration
  cle_secrete
    Field required
```

Vous avez déjà créé un `.env` à la main, avec seulement `DATABASE_URL=sqlite:///./jeux.db`. Et vous écrivez à Samir, dans le chat de l'équipe :

@samir tu peux m'envoyer ton .env ? le mien marche pas

Ce message a deux défauts. Samir ne sait pas ce qui bloque. Et ce qui est demandé pose un problème en soi : relisez « Deux conséquences », en séance 1, partie 2.

2.1 Chacun réécrit ce message, seul, sur papier ou dans un fichier hors du dépôt : il n'est pas noté. Il suit les cinq rubriques de la partie 3 : ce que j'essaie de faire, ce qui bloque (l'erreur, copiée), ce que j'ai déjà essayé, mon hypothèse, ce dont j'ai besoin (de qui, pour quand).

2.2 Échangez avec un voisin : chacun vérifie le message de l'autre avec la liste ci-dessous.

Si vous avez le temps : même exercice avec « help docker marche pas ». Inventez les détails qui manquent, entre crochets : `[l'erreur exacte]`.

Vérification : le message contient les cinq rubriques, un destinataire et une échéance, et il ne demande aucun secret.

Après la pause : l'écran bleu

Le 19 juillet 2024, des millions de PC Windows affichent un écran bleu, redémarrent, et replantent. Aéroports à l'arrêt, vols annulés, hôpitaux, banques, chaînes de télé. Vous l'avez peut-être vu aux infos, ou vécu dans un aéroport.

La cause : une mise à jour de **CrowdStrike**, un logiciel de sécurité installé dans les entreprises, envoyée à tous ses clients en même temps. Environ **8,5 millions de machines** tombent.

Cette mise à jour avait été vérifiée... par un seul outil automatique, qui avait lui-même un bug : il l'a laissée passer. Derrière, personne ne l'a révérifiée, parce que les mises à jour du même type s'étaient toujours bien passées. Et elle est partie chez tout le monde d'un coup, sans d'abord l'essayer sur quelques machines.

Aucun développeur n'a voulu casser quoi que ce soit. Il a manqué **un regard de plus** avant d'envoyer : quelqu'un d'autre que l'auteur, qui relit, qui lance, qui dit « attends ». C'est exactement le rôle d'une pull request relue.

La démo

Voici le genre de ligne qu'un relecteur doit arrêter. L'enseignant projette une route qui cherche un utilisateur par email, écrite comme on ne devrait jamais l'écrire :

```
@routeur.get("/utilisateurs/recherche")
def rechercher(email: str, session: SessionDep):
    requete = text(f"SELECT * FROM utilisateurs WHERE email = '{email}'")
    return [dict(ligne._mapping) for ligne in session.execute(requete)]
```

D'abord, un appel normal :

```
curl -G http://127.0.0.1:8000/utilisateurs/recherche --data-urlencode "email=lea@example.com"
```

Puis un email qui n'en est pas un :

```
curl -G http://127.0.0.1:8000/utilisateurs/recherche --data-urlencode "email=' OR '1'='1'"
```

La condition `email = '' OR '1'='1'` est toujours vraie : **toute la table des utilisateurs sort**. Le `f"..."` ne colle pas une donnée, il colle du **code SQL** que l'utilisateur a écrit.

Le défi

Cinq pull requests attendent une relecture sur le dépôt modèle, #2 à #6. **L'une d'elles a exactement cette faille**, et permet de lire les mots de passe de tous les utilisateurs.

Ne la cherchez pas maintenant : c'est l'exercice 3. Juste après, chaque groupe annonce laquelle, et la ligne exacte.

Partie 4 : relire, ce qu'on regarde

À quoi sert la relecture

Trouver les défauts avant `main`. Partager la connaissance : après la relecture, deux personnes connaissent ce code.

Elle ne sert **pas** à montrer qu'on est plus fort, ni à imposer ses goûts.

Dans quel ordre

1. **L'intention** : la description, l'issue.
2. **La vue d'ensemble** : fichiers touchés, taille, tests, CI.
3. **Le détail**, ligne à ligne.
4. **La décision** (partie 6).

Une PR de 900 lignes sans test se juge dès l'étape 2.

La liste

Catégorie	Questions
Correction	Fait-elle ce que l'issue demande ? Liste vide, <code>None</code> , zéro, doublon ?
Sécurité	Entrée utilisateur dans du SQL ? Propriété vérifiée ? Donnée sensible qui sort ? Secret en dur ?
Performance	Requête dans une boucle ? Table entière chargée ? Liste sans pagination ?

Catégorie	Questions
Lisibilité	Noms clairs ? Fonctions trop longues ? Code mort ? Couches respectées ?
Tests	Échoueraient-ils sans le changement ? Un test a-t-il été modifié ou supprimé ?
Documentation	Le README et les ADR sont-ils encore justes ?

Ce qu'on ne regarde pas

Ce qu'un outil vérifie. Formatage, ordre des imports : `ruff` s'en charge.

Ses goûts. « J'aurais écrit une boucle » n'est pas un défaut.

Les lignes rouges

On relit naturellement ce qui est ajouté, en vert. Les défauts graves se cachent souvent dans ce qui est **supprimé**, en rouge :

```
class Pagination:
    def __init__(self, saut: int = 0, limite: int = 20):
        self.saut = max(0, saut)
-       self.limite = min(max(1, limite), 100)
+       self.limite = max(1, limite)
```

Une ligne rouge, et n'importe quel client demande un million de jeux en une requête.

Méfiez-vous de trois signaux : un commit « simplifie » ou « nettoyage » qui change un comportement, un test modifié pour qu'il passe, une configuration changée « parce que c'est plus pratique ».

Pour aller plus loin : lancer le code d'une PR

Le diff ne suffit pas toujours. Pour une PR du dépôt modèle, clonez-le à côté du vôtre, dans un terminal où l'environnement virtuel de votre dépôt est activé :

```
git clone https://github.com/<enseignant>/api-jeux-reprise.git atelier
cd atelier
cp ../api-jeux-<groupe>/.env .           # Windows : copy ../api-jeux-<groupe>\.env .
git switch <branche-de-la-pr>
python scripts/peupler.py                # le catalogue de démo, dans la base de ce dossier
ECHO_SQL=true fastapi dev app/main.py    # Windows (PowerShell) : $env:ECHO_SQL="true"; fastapi
dev app/main.py
```

`ECHO_SQL=true` affiche chaque requête SQL dans le terminal. C'est le moyen le plus rapide de voir un N+1.

Partie 5 : écrire un commentaire de relecture

Le problème

C'est nul.

Pourquoi t'as fait ça ???

À l'écrit, sans voix ni visage, le lecteur imagine le ton. Presque toujours le pire.

L'auteur se défend au lieu de corriger, et la PR s'enlise.

Les principes

Le code, pas la personne. « Cette requête charge toute la table », pas « tu charges toute la table ».

Pourquoi. La conséquence concrète, pas seulement le diagnostic.

Une piste. Un exemple de code, un lien vers la documentation.

Bloquant ou non. C'est l'information la plus utile pour l'auteur.

Ce qui est réussi. « Bonne idée de tester le catalogue vide » dit à l'auteur quoi continuer à faire.

Les étiquettes

On préfixe chaque commentaire par une étiquette, selon la convention *Conventional Comments* :

Étiquette	Sens	Bloquant
<code>issue:</code>	un problème à corriger	oui
<code>suggestion:</code>	une amélioration	à préciser
<code>question:</code>	besoin de comprendre	non
<code>nit:</code>	un détail	non
<code>thought:</code>	une idée pour plus tard, hors de cette PR	non
<code>praise:</code>	ce qui est réussi	non

Au besoin, on précise : `suggestion (bloquant):` .

Avant, après

Sécurité

~~Attention sécurité:~~

issue: cette ligne écrit tout le formulaire de connexion dans le journal, mot de passe compris :

`logger.info("connexion %s", formulaire)` . Quiconque lit les journaux obtient des mots de passe en clair. `formulaire.username` suffit.

Performance

~~C'est lent:~~

question: `similaires` charge tous les jeux, puis filtre le genre en Python. Avec 5 000 jeux, chaque appel transfère toute la table. Un `.where(Jeu.genre == jeu.genre)` dans la requête ne suffirait pas ?

Lisibilité

~~Nom pourri:~~

nit (non bloquant): `d` ne dit pas ce qu'il contient. `jeux_filtres` éviterait de remonter trois lignes pour comprendre.

Partie 6 : décider

Verdict	Quand	Effet
Comment	des remarques ou des questions, sans verdict	aucun
Approve	la PR peut partir ; les <code>nit</code> restent au choix de l'auteur	compte comme approbation
Request changes	au moins un point bloquant	l'auteur ne fusionne pas avant d'avoir corrigé

Request changes seulement s'il y a un point bloquant. Bloquer une PR pour trois `nit`, c'est bloquer pour des détails.

Les boutons de GitHub pour relire, et le rôle de l'auteur qui répond, sont pour la séance 3, sur votre propre PR.

✓ Exercice 3 : le défi (35 min)

À relire avant : parties 4 à 6.

Les PR #2 à #6 du dépôt modèle ont été écrites par cinq développeurs fictifs. **Une seule contient une injection SQL**, la même faille que dans la démo : elle permet de lire les mots de passe de tous les utilisateurs. Trouvez-la.

Leurs conversations sont verrouillées : personne ne voit les relectures des autres groupes. Vous lisez sur GitHub, et vous écrivez dans un fichier Markdown, sur ce modèle :

```
## PR #3 : relecture de Chloé

**Verdict** : Request changes

- `app/dépôts/jeux.py`, ligne 51 : **question:** ...
- `app/modèles/jeux.py`, ligne 158 : **nit:** ...
```

3.1 Répartissez les cinq PR entre les membres : chaque PR a au moins un lecteur.

3.2 Pour chaque PR, lisez dans l'ordre de la partie 4 : la description, les fichiers modifiés, puis le diff ligne à ligne, **lignes rouges comprises**. Cherchez l'endroit où une donnée envoyée par l'utilisateur arrive dans une requête SQL.

3.3 Une fois la PR trouvée, écrivez le commentaire que vous laisseriez sur la ligne fautive : le fichier et la ligne, l'étiquette, **bloquant ou non**, pourquoi c'est grave, et comment corriger (partie 5). Ajoutez le verdict (partie 6).

3.4 Gardez votre réponse pour vous : vous l'annoncerez juste après, dans « On présente ».

Vérification : le groupe a une PR, un fichier, une ligne, et un commentaire qui explique le risque et la correction.

Si vous avez fini : pour aller plus loin

Les autres PR ne sont pas propres non plus. Notez un problème par PR, avec son étiquette.

- une seule de ces PR mérite d'être approuvée ;
- plusieurs contiennent un problème de sécurité, pas toujours là où on l'attend ;
- l'une a une CI rouge : lisez pourquoi, puis relisez quand même le reste ;
- la plus longue n'est pas forcément la plus dangereuse, mais c'est celle où un défaut se cache le mieux.

En cas de doute sur une PR, lancez son code : voir « Lancer le code d'une PR », en partie 4.

On présente : la révélation du défi

Chaque groupe annonce, à tour de rôle, la PR qui contient l'injection, avec le fichier et la ligne.

Puis **un premier groupe**, désigné par l'enseignant, projette son commentaire de relecture sur la ligne fautive. La classe le valide avec cette grille :

Critère	Rempli ?
Le commentaire désigne le fichier et la ligne	
Il explique pourquoi c'est grave, avec une conséquence concrète	
Il propose une correction	
Il est étiqueté et dit qu'il est bloquant	
Le verdict est <i>Request changes</i>	

L'enseignant révèle alors la **#2**, et rejoue l'injection au vidéoprojecteur, avec la requête de démonstration du corrigé. La même faille que dans la démo d'après la pause : une donnée utilisateur collée dans du SQL par une f-string.

Quiz des issues, s'il reste du temps

Six questions sur les séances 1 et 2, projetées. On joue **par groupe de projet** : pour chaque question, votre groupe a 45 secondes pour se mettre d'accord, puis lève sa réponse : la lettre, écrite en grand sur une feuille. L'enseignant révèle la réponse et son explication.

Ce quiz ne compte pas dans la note. Mais le quiz final de la séance 4, qui rapporte jusqu'à + 2 à votre groupe, tire ses questions du même lot : c'est le meilleur entraînement.

À retenir

Le backlog

Évolution : des critères d'acceptation vérifiables par oui ou par non.

Une étiquette pour filtrer, une assignation quand on commence.

On découpe par résultat, pas par fichier.

`Closes #N`, en anglais, pour fermer à la fusion.

Le blocage

Au-delà de 30 à 60 minutes sans progrès, on signale.
Objectif, erreur, essais, hypothèse, besoin.
Dans l'issue, pas en privé.

La relecture

Intention, vue d'ensemble, détail, décision.
Correction, sécurité, performance, lisibilité, tests.
Les lignes rouges comptent plus que les vertes.
Ce qu'un outil vérifie, un humain ne le commente pas.

Le commentaire et la décision

Sur le code, pas sur la personne. Pourquoi, et une piste.
Étiqueté, bloquant ou non.
Request changes seulement pour un point bloquant.

Les erreurs qui coûtent le plus de temps

Symptôme	Cause
Deux personnes corrigent le même bug	Personne ne s'est assigné l'issue
<code>Corrige #12</code> ne ferme pas l'issue	Mot-clé en français : GitHub attend <code>Closes</code>
Une tâche n'avance plus depuis trois jours	Blocage jamais signalé
Une faille passe malgré la relecture	Seules les lignes vertes ont été relues
Un test supprimé passe inaperçu	Le relecteur s'est fié à la CI verte
La discussion tourne au conflit	Remarques sur la personne

Prochaine séance

Séance 3, lundi : la pull request des statistiques, puis le README. Vous corrigez l'issue des statistiques par une PR exemplaire, relue jusqu'à la fusion. Après la pause, vous écrivez le README de votre dépôt.
Pour arriver prêt : tout le backlog est saisi en issues, et l'issue des statistiques est assignée à celui qui la corrigera.